Deep Dive — Module 10: How Everything Connects Together

> ***Goal:*** *assemble the nine components into one mental model. Trace a real upload and a real search end-to-end, see exactly where each technology does its job, and understand the cross-cutting concerns (security, auditing, errors) that run through every flow.*

Where it sits: This is the **integration view**. Modules 1–9 each taught one technology in isolation; here they become a system. If you can narrate the two flows below from memory, you understand the project.

---

1. The cast (one line each)

```
React (3000)       UI: forms, results, dashboards; calls the API with the JWT
Spring Boot (8080) Orchestrator: parses, chunks, embeds, stores, secures, audits
Embedding (5000)   Python/Flask: text → 384-number vector (all-MiniLM-L6-v2)
Qdrant (6333)      Vector DB: stores chunk vectors; finds nearest neighbours
PostgreSQL (5432)  System of record: metadata, users, audit logs, alerts
PDFBox / POI       Extract text from PDF / DOCX (in Spring Boot)
Tesseract (Tess4J) OCR fallback for scanned images (in Spring Boot)
JWT / Spring Sec.  Who you are + what you may do, on every request
Docker Compose     Runs all of the above together, reproducibly
```

The backend is the hub: **everything flows through Spring Boot**, which calls the embedding service, Qdrant, and PostgreSQL, and applies security and auditing around each operation.

---

2. Complete upload flow

```
USER (clerk Priya) selects invoice_acme_may2026.pdf → clicks Upload
  │
REACT  builds FormData(file); POST /api/documents/upload
        Header: Authorization: Bearer <JWT>;  Body: multipart PDF
  │
SPRING BOOT
  ├─ JwtAuthenticationFilter: verify signature + expiry → role=ROLE_CLERK ✅    [Module 8]
  ├─ DocumentController.upload(MultipartFile) → DocumentService.handleUpload()  [Module 4]
  │   (@Transactional begins)                                                    [Module 1 §11]
  ├─ 1. File-type detect: ".pdf" → PDFBox                                        [Module 9]
  ├─ 2. Extract text: PDFTextStripper.getText() → "INVOICE Acme Corp INV-2048…"
  │      if empty (scanned) → Tesseract OCR; set ocr_used=true                   [Module 7]
  ├─ 3. Clean text (normalize whitespace/noise)
  ├─ 4. Chunk ~800 chars → 3 chunks                                             [Module 9]
  ├─ 5. Embed each chunk: POST embedding:5000/embed → [384 floats] × 3          [Module 5]
  ├─ 6. Upsert 3 points to Qdrant (vector + payload{document_id, vendor, chunk_index}) [Module 2
  ├─ 7. Persist to PostgreSQL: documents row (status=INDEXED), chunk rows, audit row   [Module 1
  │   (@Transactional commits — document + audit together, all-or-nothing)
  └─ Return 202 { documentId, status:"INDEXED", chunksGenerated:3 }
  │
REACT  shows "Upload complete ✅"; processing stepper all green
```

What each technology contributed: React captured the file; JWT proved the clerk's identity; PDFBox (or OCR) produced text; the chunker split it; Python turned chunks into vectors; Qdrant stored them; PostgreSQL recorded the facts and the audit trail; the transaction guaranteed consistency; Docker is what made all five services reachable.

---

3. Complete search flow

```
USER uploads a query invoice → clicks "Search Similar"
  │
REACT  POST /api/documents/search (multipart query PDF) + Bearer JWT
  │
SPRING BOOT
  ├─ JWT validated (all roles may search)                                    [Module 8]
  ├─ Same front end: extract → clean → chunk → embed → 3 query vectors  [Modules 9,5]
  ├─ For each query vector: POST qdrant:6333/.../points/search {vector, top:5} [Module 2]
  │     ← per chunk: [{document_id, score, payload}, …]
  ├─ MERGE by document_id, take MAX score per document
  │     doc-a3f9c1b2 → 0.94   doc-b4e2f1a0 → 0.87   doc-c5d3g2b1 → 0.71
  ├─ THRESHOLD filter (≥ 0.60/0.70); map scores → labels (STRONG/RELATED/WEAK) [Module 2 §10]
  ├─ DUPLICATE rule: score ≥ 0.90 AND matching invoice_number → DUPLICATE alert [Modules 1,2]
  ├─ AUDIT: insert search log + results + audit row                          [Module 1]
  └─ Return { results:[…ranked…], duplicateAlert:true }
  │
REACT  renders ranked ResultCards with score rings; red DUPLICATE banner if flagged
```

The search reuses the *entire* upload front-end (extract→chunk→embed) — the only difference is the vectors are used to **query** Qdrant rather than to store. This symmetry is why the system finds semantic matches: the query and the corpus are embedded by the *same model* into the *same space*.

---

4. Cross-cutting concerns (present in every flow)

- **Security (JWT + RBAC):** every request is authenticated by the filter; roles gate endpoints; clerks are further limited to their department in the service layer. [Module 8]
- **Auditing:** login, upload, search, delete, and alert actions all write to the append-only `audit_logs` table — immutable, retained for years. [Module 1 §13]
- **Transactions:** multi-write operations (document + audit, search log + results) commit atomically. [Module 1 §11]
- **Error handling:** a global `@RestControllerAdvice` maps failures to the consistent error shape (`EXTRACTION_FAILED` 422, `UPSTREAM_ERROR` 502, etc.) with no stack traces leaked. [Module 4 §10]
- **Observability:** structured JSON logs (requestId/userId/action/durationMs) + Actuator/Prometheus metrics; `/health` aggregates Qdrant/Postgres/embedding status. [Module 4]

- **Infrastructure:** Docker Compose runs the five services on one network; they reach each other by service name; databases use volumes. [Module 6]

---

5. The two databases, side by side

	PostgreSQL	Qdrant
Stores	Facts: metadata, users, audit, alerts	Meaning: 384-dim chunk vectors + payload
Answers	"What are the exact details / history?"	"What is *similar* to this?"
Lookup	Exact match, ranges, joins	Nearest-neighbour by cosine
Integrity	ACID, constraints, immutable audit	Eventual; rebuildable from source docs
If lost	Catastrophic (lose the record)	Recoverable (re-embed)

They cooperate: a search hits **Qdrant** for candidate document_ids + scores, then often joins to **PostgreSQL** for display metadata, and the duplicate rule needs *both* (semantic score from Qdrant, invoice number from PostgreSQL).

---

6. End-to-end mental checklist

For any feature, you should be able to answer: *Who is allowed (JWT/RBAC)? → How does text get out of the file (PDFBox/POI/OCR)? → How is it chunked and embedded (chunker/Python)? → Where do vectors go or get matched (Qdrant)? → What facts are recorded (PostgreSQL)? → What's audited? → How are errors surfaced? → Where does it run (Docker)?* If every arrow has an owner, you've got the full picture.

---

7. Common integration pitfalls

- **Dimension mismatch across services** — model output (384) must equal Qdrant size and pgvector width; changing the model means re-embedding everything.
- **`localhost` between containers** — use service names on the Compose network. [Module 6]

- **Partial writes** — forgetting `@Transactional` can leave a document with no audit row (or vice versa).
- **Not grouping chunk results by document_id** — duplicates in the result list. [Module 2]
- **Hung embedding/Qdrant call** — set timeouts; map failures to `502 UPSTREAM_ERROR`. [Module 4]
- **Frontend-only access control** — enforce RBAC in the backend, always. [Module 8]
- **Search/index using different models or cleaning** — query and corpus must be embedded identically or scores are meaningless.

---

8. Practice exercises

1. Without looking, write the upload flow as a numbered list naming the technology at each step.
2. Do the same for the search flow; mark the one point where it diverges from upload.
3. For each of the 9 components, write one sentence: "If this were removed, the system would…".
4. Trace a *scanned* invoice upload: which extra step runs, and which flag is set?
5. Trace a duplicate detection: list the exact signals from Qdrant and from PostgreSQL that combine to fire the alert.
6. Draw the Docker Compose network and label which service talks to which (and on what port).
7. Pick three failure points (embedding down, Qdrant down, bad file) and state the HTTP status and error code each should produce.
8. Explain why query and corpus must use the same embedding model, using the idea of a shared vector space.

---

9. Self-check questions

- Why does *everything* route through Spring Boot rather than React talking to Qdrant directly?
- What is identical between the upload and search pipelines, and what differs?
- How do the two databases divide responsibility, and where do they cooperate?

- Which cross-cutting concerns appear in *every* request?
- What single change forces re-embedding the whole corpus, and why?
- Where is consistency (atomicity) guaranteed, and what breaks without it?
- How does the duplicate-invoice alert use both databases?

---

10. Where to go next

You've now covered every component. Strong next steps: (1) stand the stack up with `docker-compose up` and watch the logs during a real upload; (2) re-read the TRD's sequence diagrams alongside §2–§3 here; (3) implement one vertical slice (upload → search) end-to-end yourself, even with stubbed pieces, to feel how the parts connect.

Navigation: ← Module 9 PDFBox & POI | Index | — (end)